CO1 – Assessment Tool 1

Scalability Audit of an E-Commerce Platform

CSA06 – Design and Analysis of Algorithms

NAME:LAKSHMIPATHI N

REG NO:192424124

1. Aim

To audit five back-end modules of the ShopSphere platform, derive their exact operation counts and asymptotic complexity, validate every derivation by running code that counts the basic operation, and recommend which modules must be redesigned before the catalogue grows to 1,000,000 products.

2. Objective

1. Identify the input size and basic operation of each module and predict how its operation count grows.
2. Derive exact comparison and operation counts for the iterative modules M1 and M2, and test the claim that M2 is $\Theta(n \log n)$.
3. Solve the recurrence of search version S2 by backward substitution and state time and space complexity (iterative and recursive).
4. Apply the Master Theorem to M4 (current and redesigned) and the extended Master Theorem or recursion tree to M5.
5. Analyse best, average and worst cases of S1 and S2, including S1's average for a probability p that the product exists.
6. Prove formally that $f(n) = 3n^2 + 10n \log n + 50$ is $\Theta(n^2)$ and not $O(n)$.
7. Estimate running times at n = 1,000,000 against a 10-minute nightly window and recommend redesigns.

3. Assumptions and Method

- **Server speed.** The question paper states that servers execute 10 basic operations per second; the exponent is lost in the PDF text, so **10^9 operations per second** is assumed. Times are also given at 10^8 operations per second as a sensitivity check. Operation counts and complexity classes do not depend on this assumption.
- **Nightly window.** 10 minutes = 600 s = 6×10^{11} basic operations at 10^9 operations per second.
- **Logarithms** are to base 2. Closed forms for recurrences are exact for $n = 2^k$ and accurate asymptotically otherwise.
- **Method.** (i) derive each count mathematically; (ii) implement each module with a counter on its basic operation; (iii) check that the measured count equals the formula; (iv) project to $n = 10^5$ and $n = 10^6$.

4. Algorithm and Analysis

4.1 Input size and basic operation (part a)

Module	Input size n	Basic operation	Where it occurs	Expected growth
M1 Duplicate detector	Number of products	Title comparison Product[i].title = Product[j].title	Innermost loop	Once per pair of products, so quadratic: doubling n quadruples the work
M2 API rate limiter	Number of requests	Audit(Request[j])	Inner FOR loop	Inner loop lengths are 1, 2, 4, … (geometric series), so the total is dominated by the last round: linear
M3 Search S1 / S2	Number of products	Key comparison with an entry	Each scan step (S1) / each halving (S2)	S1 up to n; S2 one comparison per halving, about $\log_2 n$
M4 Recommendation engine	Size of customer–product data	One unit of merge work per element (the + n term) at every recursion node	After the recursive calls	Decided by the number of nodes: 4 calls give n^2 leaf work, 2 calls give n log n
M5 Sales analytics	Number of sales records	Comparison while sorting a sub-array at each node	Sort at every recursion level	About n(log n − i) per level, over log n levels: about ½ n $\log^2$ n

4.2 M1 – Duplicate-listing detector (part b)

For i = 0 … n−2 the inner loop runs (n − 1 − i) times, so

C(n) = (n−1) + (n−2) + … + 2 + 1 = **n(n−1)/2**.

Every pair is always compared, so best, average and worst cases are identical.

- n = 100,000: 4,999,950,000 comparisons
- n = 1,000,000: 499,999,500,000 (about 5×10^{11})

Since $n^2/4 \le n(n-1)/2 \le n^2$ for $n \ge 2$, **time = $\Theta(n^2)$**. Only the counters i and j are needed, so **auxiliary space = $\Theta(1)$**.

4.3 M2 – API rate limiter (part b)

The outer loop sets i = 1, 2, 4, …, 2^(k−1) and stops at the first $i = 2^k \ge n$, so it runs $k = \lceil \log_2 n \rceil$ times. In the round where $i = 2^t$ the inner loop runs $i = 2^t$ times (not n times):

$A(n) = 2^0 + 2^1 + \dots + 2^{(k-1)} =$ **$2^{\lceil \log_2 n \rceil} - 1$**.

Because $n \le 2^k < 2n$, we get **$n - 1 \le A(n) \le 2n - 2$**, so A(n) = Ω(n) and O(n): the tight bound is **$\Theta(n)$**.

The team member's claim is incorrect. Bounding every inner loop by n gives only the loose upper bound k·n. The inner loops grow geometrically, so their sum is dominated by the final term. Θ(n log n) is not a tight bound, because n log n is not O(n). Time is Θ(n) and space is Θ(1).

Verification for n = 1,024 (= 2^{10}). i takes the values 1, 2, 4, …, 512 (i = 1024 is not less than 1024), so there are 10 rounds:

A = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512 = **1,023 = $2^{10} - 1$**, and n − 1 = 1,023. The claimed $n \log_2 n = 10{,}240$ overstates the work tenfold.

4.4 S2 – Binary search recurrence by backward substitution (part c)

T(n) = T(n/2) + 1, T(1) = 1. The method applies because each step shrinks n by a constant factor with constant extra work. Take $n = 2^k$:

```
T(n) = T(n/2) + 1
     = [T(n/4) + 1] + 1  = T(n/4) + 2
     = [T(n/8) + 1] + 2  = T(n/8) + 3
     ...
     = T(n/2^i) + i
```

The recursion ends when $n/2^i = 1$, i.e. $i = \log_2 n$, and T(1) = 1. Therefore **$T(n) = \log_2 n + 1$**. For general n, **$T(n) = \lfloor \log_2 n \rfloor + 1$** (proved by induction using $\lfloor \log_2 \lfloor n/2 \rfloor \rfloor = \lfloor \log_2 n \rfloor - 1$). For n = 1,000,000 this is 20 comparisons.

Implementation	Time	Space
Iterative (lo, hi loop)	Θ(log n)	Θ(1)
Recursive	Θ(log n)	Θ(log n): call stack of $\lfloor \log_2 n \rfloor + 1$ frames (20 frames for 10^6 records)

4.5 Best, average and worst cases of S1 and S2 (part e)

S1 – sequential scan of the unsorted catalogue

- Best: product is first, 1 comparison.
- Worst: product is last or absent, n comparisons.
- Average when present: each position has probability 1/n, so $(1/n) \cdot \Sigma i = (n+1)/2$.

- With probability p that the product exists: **$A(n, p) = p(n+1)/2 + (1-p)n$**.
- For n = 1,000,000 and p = 0.5: A = 0.5 × 500,000.5 + 0.5 × 1,000,000 = **750,000.25 comparisons**.

S2 – binary search of the sorted index

- Best: key is the middle entry, 1 comparison.
- Worst: key absent or at the deepest level, $\lfloor \log_2 n \rfloor + 1 = 20$ comparisons for $n = 10^6$.
- Average (key present): about $\log_2(n+1) - 1 \approx 18.93$, because half of all entries lie on the deepest level of the implicit decision tree.

n = 1,000,000	Best	Average	Worst
S1	1	500,000.5 (present); 750,000.25 at p = 0.5	1,000,000
S2	1	about 18.93	20

4.6 M4 – Recommendation engine, Master Theorem (part d)

For T(n) = aT(n/b) + f(n), compare f(n) with n^(log_b a). Case 1: f polynomially smaller gives Θ(n^(log_b a)). Case 2: f = Θ(n^(log_b a)) gives Θ(n^(log_b a) log n). Case 3: f polynomially larger (with regularity) gives Θ(f).

	Current design	Proposed redesign
Recurrence	$T(n) = 4T(n/2) + n$	$T(n) = 2T(n/2) + n$
a, b, f(n)	4, 2, n	2, 2, n
n^(log_b a)	n^2	n
Comparison	$n = O(n^{(2-\varepsilon)})$, $\varepsilon = 1$	$n = \Theta(n)$
Case	1	2
Complexity	**$\Theta(n^2)$**	**$\Theta(n \log n)$**
Exact solution (recursion tree)	$2n^2 - n$	$n \log_2 n + n$
Operations at $n = 10^6$	2×10^{12}	2.09×10^7
Effect of doubling the data	$T(2n)/T(n) \to 4$ (four times slower)	$T(2n)/T(n) = 2(\log n + 2)/(\log n + 1) \approx 2.1$

Removing the replication cuts the recursive calls from 4 to 2, which moves the dominant cost from the n^2 leaves to n per level over log n levels. Doubling the catalogue to 2×10^6 would take the current design from 33 minutes to about 2.2 hours.

4.7 M5 – Sales analytics engine (part d)

$T(n) = 2T(n/2) + n \log n$ gives a = 2, b = 2 and n^(log_b a) = n.

- Case 1 needs f = O(n^(1−ε)): n log n is larger than n, so it fails.
- Case 2 (basic form) needs f = Θ(n): n log n is not Θ(n), so it fails.
- Case 3 needs f = Ω(n^(1+ε)) for some ε > 0: n log n = o(n^(1+ε)) for every ε > 0, so it fails.

None of the three standard cases applies. The extended Master Theorem states that if f(n) = Θ(n^(log_b a) log^k n) with k ≥ 0, then T(n) = Θ(n^(log_b a) log^(k+1) n). Here k = 1, so **$T(n) = \Theta(n \log^2 n)$**.

Recursion-tree check ($n = 2^L$, $L = \log_2 n$). At depth i there are 2^i nodes of size $n/2^i$, each costing $(n/2^i)\cdot\log(n/2^i)$, so each level costs $n(L - i)$. Summing:

$\Sigma\, n(L - i)$ for $i = 0 \ldots L-1 = n\cdot L(L+1)/2$, plus n for the leaves, giving **$T(n) = n\cdot L(L+1)/2 + n = \Theta(n \log^2 n)$**.

Sorting at every level wastes a logarithmic factor; merging already-sorted halves would give Θ(n log n).

4.8 Formal proof that $f(n) = 3n^2 + 10n \log n + 50$ is $\Theta(n^2)$ (part f)

For $n \geq 1$ we have $0 \leq \log n \leq n$.

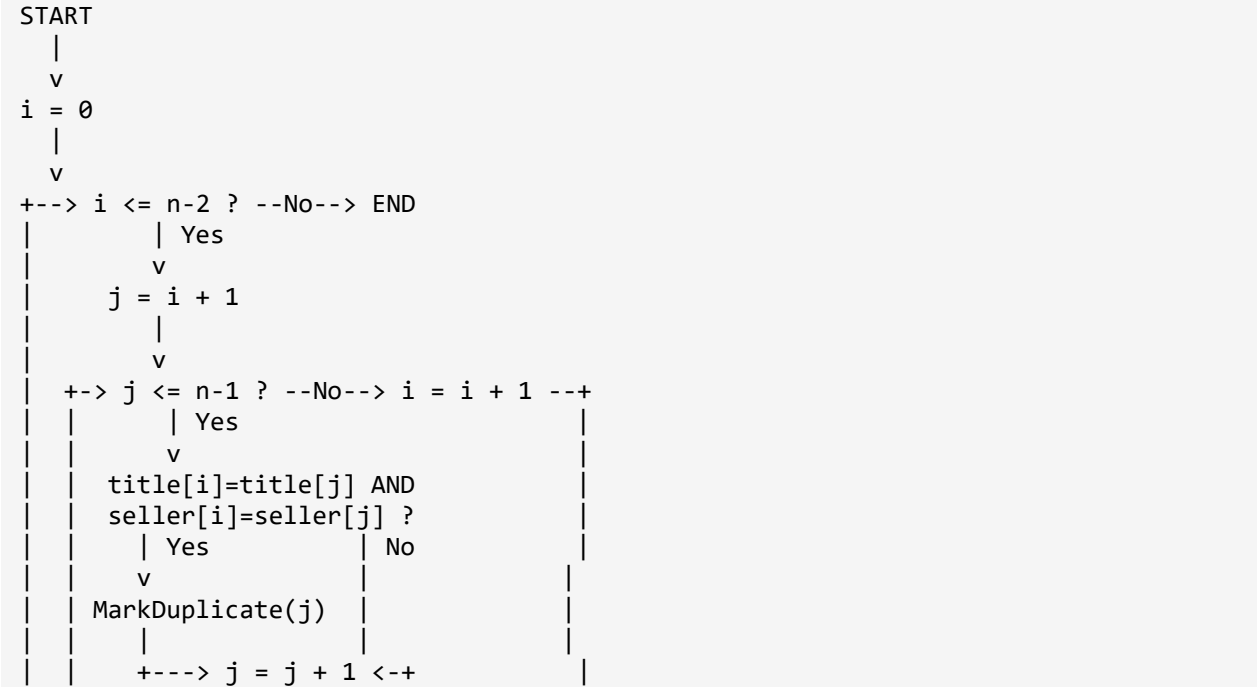
- **Lower bound.** All terms are non-negative, so $f(n) \geq 3n^2$. Take $c_1 = 3$ and $n_0 = 1$.
- **Upper bound.** $10n \log n \leq 10n^2$ and $50 \leq 50n^2$, so $f(n) \leq 3n^2 + 10n^2 + 50n^2 = 63n^2$. Take $c_2 = 63$ and $n_0 = 1$. (A tighter pair is $c_2 = 4$ with $n_0 = 64$.)

Hence $3n^2 \leq f(n) \leq 63n^2$ for all $n \geq 1$, so **$f(n) = \Theta(n^2)$**.

f(n) is not O(n). Suppose $f(n) \leq c\cdot n$ for some $c > 0$ and all $n \geq n_1$. Since $f(n) \geq 3n^2$, this requires $3n^2 \leq cn$, i.e. $n \leq c/3$, for every $n \geq n_1$. Choosing $n = \max(n_1, \lceil c/3 \rceil + 1)$ gives a contradiction. Therefore $f(n) \notin O(n)$.

5. Flowchart

M1 – Duplicate-listing detector



```
|  +-----------+                       |
+--------------------------------------+
```

M2 – API rate limiter

```
START --> i = 1
  |
  v
+--> i < n ? --No--> END
|      | Yes
|      v
|   j = 0
|      |
|      v
|  +-> j <= i-1 ? --Yes--> Audit(Request[j]) ; j = j + 1 --+
|  |      | No                                             |
|  |      v                                                |
|  |   i = i * 2                                           |
|  +-------------------------------------------------------+
+-----------+  (return to test i < n)
```

S2 – Binary search (recursive)

```
START: BS(A, key, lo, hi)
  |
  v
lo > hi ? --Yes--> return NOT FOUND
  | No
  v
mid = (lo + hi) / 2
  |
  v
A[mid] = key ? --Yes--> return mid
  | No
  v
key < A[mid] ? --Yes--> BS(A, key, lo, mid-1)
  | No
  v
BS(A, key, mid+1, hi)
```

6. Code

All modules are implemented in Python with counters on the basic operation (files: modules.py, experiment.py, test_modules.py). Key routines:

```
def m1_bruteforce(products):            # basic operation: title comparison
    n = len(products); comps = 0; marked = set()
    for i in range(n - 1):
        ti, si = products[i]
        for j in range(i + 1, n):
            comps += 1
            tj, sj = products[j]
            if ti == tj and si == sj:
                marked.add(j)
    return marked, comps                 # comps == n(n-1)/2

def m2_audit(n):                        # basic operation: Audit()
```

```
    ops, iters, i = 0, 0, 1
    while i < n:
        for _ in range(i):
            ops += 1
        i *= 2; iters += 1
    return ops, iters                    # ops == 2**ceil(log2 n) - 1

def s2_binary_recursive(arr, key, lo=0, hi=None):
    if hi is None: hi = len(arr) - 1
    if lo > hi: return -1, 0              # base case
    mid = (lo + hi) // 2
    if arr[mid] == key: return mid, 1
    if key < arr[mid]:
        idx, c = s2_binary_recursive(arr, key, lo, mid - 1)
    else:
        idx, c = s2_binary_recursive(arr, key, mid + 1, hi)
    return idx, c + 1                    # one comparison + one half-size call

def m1_hash(products):                   # proposed redesign: expected O(n)
    seen, marked = set(), set()
    for j, p in enumerate(products):
        if p in seen: marked.add(j)
        else: seen.add(p)
    return marked
```

The recurrences of M4 and M5 are executed as real recursion with counters and compared with the closed forms $2n^2 - n$, $n \log_2 n + n$ and $n \cdot L(L+1)/2 + n$.

7. Output

Test run: all 7 tests pass (M1 count and agreement with the hash version, M2 exact for every n < 3,000, S1 and S2 exhaustive, closed forms of M4 and M5, constants of the $\Theta(n^2)$ proof).

Measured counts equal the derived formulas

n	M1 = n(n−1)/2	M2 = 2^⌈log n⌉ − 1	M4 current = $2n^2$ − n	M4 redesign = n log n + n
2	1 = 1	1 = 1	6 = 6	4 = 4
4	6 = 6	3 = 3	28 = 28	12 = 12
512	130,816 = 130,816	511 = 511	523,776 = 523,776	5,120 = 5,120

M2 checks: n = 1,024 gives 1,023 operations in 10 rounds; n = 1,025 gives 2,047 in 11 rounds; n = 10^6 gives 1,048,575 against a claimed 19,931,569.

M1 timing – brute force versus hash redesign

n	Comparisons	Brute force (s)	Hash redesign (s)
500	124,750	0.0053	0.00008

n	Comparisons	Brute force (s)	Hash redesign (s)
1,000	499,500	0.0207	0.00010
2,000	1,999,000	0.1076	0.00035
4,000	7,998,000	0.3649	0.00052
8,000	31,996,000	1.9264	0.00256

Brute-force time grows with a log–log slope of about 2.1, confirming $\Theta(n^2)$. The duplicate sets found by both versions were identical.

Search verification: S1 exhaustive at n = 2,000 gave a mean of 1,000.5 comparisons for present keys (theory 1,000.5) and 1,500.25 at p = 0.5 (theory 1,500.25). S2 over all 10^6 keys gave best 1, worst 20 and mean 18.951 (theory 18.932); the recursive and iterative versions agreed exactly.

Scalability projection (part g) – 10^9 operations per second, window 10 minutes

Module	Operations, n = 10^5	Operations, n = 10^6	Time at 10^9 ops/s	Time at 10^8 ops/s	Within 10 min?
M1 brute force	5.00×10^9	5.00×10^{11}	8.33 min	83 min	Yes at 10^9 (83% of window); No at 10^8
M2	1.31×10^5	1.05×10^6	1.05 ms	10.5 ms	Yes
S2 (worst)	17	20	20 ns	0.2 µs	Yes
M4 current	2.00×10^{10}	2.00×10^{12}	33.3 min	5.6 h	**No**
M4 redesign	1.76×10^6	2.09×10^7	20.9 ms	0.21 s	Yes
M5	1.47×10^7	2.10×10^8	0.21 s	2.1 s	Yes
Report generator f(n)	3.00×10^{10}	3.00×10^{12}	50 min	8.3 h	**No**

Largest n that fits the 10-minute window (10^9 ops/s): M1 about 1,095,000; M4 current about 548,000; report generator about 447,000; M5 about 1.3×10^9; M4 redesign about 1.7×10^{10}.

8. Result

- **M1:** exactly $n(n-1)/2$ comparisons, $\Theta(n^2)$ time and $\Theta(1)$ auxiliary space.
- **M2:** exactly $2^{\lceil \log_2 n \rceil} - 1$ operations, bounded by $n - 1 \le A(n) \le 2n - 2$, so $\Theta(n)$. The $\Theta(n \log n)$ claim is rejected; for n = 1,024 the count is 1,023.
- **S2:** $T(n) = \lfloor \log_2 n \rfloor + 1 = \Theta(\log n)$; space $\Theta(1)$ iterative and $\Theta(\log n)$ recursive.
- **M4:** current design $\Theta(n^2)$ by Master Theorem case 1; redesign $\Theta(n \log n)$ by case 2. Doubling the data multiplies time by 4 versus about 2.1.

- **M5:** no standard case applies; $\Theta(n \log^2 n)$ by the extended Master Theorem and the recursion tree.
- **S1 average** at $n = 10^6$ and $p = 0.5$: 750,000.25 comparisons.
- **Report generator:** $f(n) = \Theta(n^2)$ with $c_1 = 3$, $c_2 = 63$, $n_0 = 1$, and $f(n)$ is not $O(n)$.

Recommendations

1. **M4 – redesign (mandatory).** Adopt the $2T(n/2) + n$ design: about 10^5 times fewer operations at $n = 10^6$ (33 minutes to 21 ms).
2. **Report generator – redesign (mandatory).** Replace the quadratic $3n^2$ part with a sort-and-scan or hash aggregation.
3. **M1 – redesign (recommended).** It fits the window at 10^9 ops/s only narrowly, fails at 10^8 ops/s, and breaks at about 1.1 million products. Use a hash set keyed on (title, seller), expected $\Theta(n)$ time, $\Theta(n)$ space.
4. **M5 – optional improvement.** Merge sorted halves instead of sorting at every level to reach $\Theta(n \log n)$.
5. **M2 and S2 – retain.** Use the sorted index (S2) instead of sequential scan (S1): 20 versus about 750,000 comparisons on average.

9. Conclusion

The audit shows that the loop bounds written in code can mislead: M2 looks like n log n but is linear because its inner loops grow geometrically, whereas M1, the current M4 design and the report generator are genuinely quadratic and are the real scalability risks. The Master Theorem showed that reducing the number of recursive calls from four to two turns $\Theta(n^2)$ into $\Theta(n \log n)$, and the extended theorem together with a recursion tree resolved M5, which lies between cases 2 and 3. Running the modules with operation counters reproduced every derived formula exactly, so the recommendations follow directly from the analysis. The time estimates depend on the assumed speed of 10^9 operations per second, but the operation counts, complexity classes and the ranking of the modules do not.